10-scaling-up-road-to-the-top-part-3

May 27, 2025

[30]:
```
# install fastkaggle if not available
try: import fastkaggle
except ModuleNotFoundError:
    !pip install -Uq fastkaggle

from fastkaggle import *
```

This is part 3 of the Road to the Top series, in which I show the process I used to tackle the Paddy Doctor competition, leading to four 1st place submissions. The previous notebook is available here: part 2.

0.1 Memory and gradient accumulation

First we'll repeat the steps we used last time to access the data and ensure all the latest libraries are installed, and we'll also grab the files we'll need for the test set:

[31]:
```
comp = 'paddy-disease-classification'
path = setup_comp(comp, install='fastai "timm>=0.6.2.dev0"')
from fastai.vision.all import *
set_seed(42)

tst_files = get_image_files(path/'test_images').sorted()
```

In this analysis our goal will be to train an ensemble of larger models with larger inputs. The challenge when training such models is generally GPU memory. Kaggle GPUs have 16280MiB of memory available, as at the time of writing. I like to try out my notebooks on my home PC, then upload them – but I still need them to run OK on Kaggle (especially if it's a code competition, where this is required). My home PC has 24GiB cards, so just because it runs OK at home doesn't mean it'll run OK on Kaggle.

It's really helpful to be able to quickly try a few models and image sizes and find out what will run successfully. To make this quick, we can just grab a small subset of the data for running short epochs – the memory use will still be the same, but it'll be much faster.

One easy way to do this is to simply pick a category with few files in it. Here's our options:

[32]:
```
df = pd.read_csv(path/'train.csv')
df.label.value_counts()
```

[32]:
```
label
normal                      1764
blast                       1738
hispa                       1594
dead_heart                  1442
tungro                      1088
brown_spot                   965
downy_mildew                 620
bacterial_leaf_blight        479
bacterial_leaf_streak        380
bacterial_panicle_blight     337
Name: count, dtype: int64
```

Let's use *bacterial_panicle_blight* since it's the smallest:

[33]:
```
trn_path = path/'train_images'/'bacterial_panicle_blight'
```

Now we'll set up a `train` function which is very similar to the steps we used for training in the last notebook. But there's a few significant differences…

The first is that I'm using a `finetune` argument to pick whether we are going to run the `fine_tune()` method, or the `fit_one_cycle()` method – the latter is faster since it doesn't do an initial fine-tuning of the head. When we fine tune in this function I also have it calculate and return the TTA predictions on the test set, since later on we'll be ensembling the TTA results of a number of models. Note also that we no longer have `seed=42` in the `ImageDataLoaders` line – that means we'll have different training and validation sets each time we call this. That's what we'll want for ensembling, since it means that each model will use slightly different data.

The more important change is that I've added an `accum` argument to implement *gradient accumulation*. As you'll see in the code below, this does two things:

1. Divide the batch size by `accum`
2. Add the `GradientAccumulation` callback, passing in `accum`.

[34]:
```
def train(arch, size, item=Resize(480, method='squish'), accum=1,␣
 ↪finetune=True, epochs=3):
    dls = ImageDataLoaders.from_folder(trn_path, valid_pct=0.2, item_tfms=item,
        batch_tfms=aug_transforms(size=size, min_scale=0.75), bs=64//accum)
    cbs = GradientAccumulation(64) if accum else []
    learn = vision_learner(dls, arch, metrics=error_rate, cbs=cbs).to_fp16()
    if finetune:
        learn.fine_tune(epochs, 0.01)
        return learn.tta(dl=dls.test_dl(tst_files))
    else:
        learn.unfreeze()
        learn.fit_one_cycle(epochs, 0.01)
```

Gradient accumulation refers to a very simple trick: rather than updating the model weights after every batch based on that batch's gradients, instead keep *accumulating* (adding up) the gradients for a few batches, and them update the model weights with those accumulated gradients. In

fastai, the parameter you pass to `GradientAccumulation` defines how many batches of gradients are accumulated. Since we're adding up the gradients over `accum` batches, we therefore need to divide the batch size by that same number. The resulting training loop is nearly mathematically identical to using the original batch size, but the amount of memory used is the same as using a batch size `accum` times smaller!

For instance, here's a basic example of a single epoch of a training loop without gradient accumulation:

```
for x,y in dl:
    calc_loss(coeffs, x, y).backward()
    coeffs.data.sub_(coeffs.grad * lr)
    coeffs.grad.zero_()
```

Here's the same thing, but with gradient accumulation added (assuming a target effective batch size of 64):

```
count = 0            # track count of items seen since last weight update
for x,y in dl:
    count += len(x)  # update count based on this minibatch size
    calc_loss(coeffs, x, y).backward()
    if count>64:     # count is greater than accumulation target, so do weight update
        coeffs.data.sub_(coeffs.grad * lr)
        coeffs.grad.zero_()
        count=0      # reset count
```

The full implementation in fastai is only a few lines of code – here's the source code.

To see the impact of gradient accumulation, consider this small model:

[35]:
```
# Record memory history
torch.cuda.memory._record_memory_history()
```

[36]:
```
train('convnext_small_in22k', 128, epochs=1, accum=1, finetune=False)
```

```
/home/vscode/.local/lib/python3.10/site-packages/timm/models/_factory.py:114:
UserWarning: Mapping deprecated model name convnext_small_in22k to current
convnext_small.fb_in22k.
  model = create_fn(
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

Let's create a function to find out how much memory it used, and also to then clear out the memory for the next run:

[37]:
```
import gc
def report_gpu():
    print(f"Peak Memory Requirement: {torch.cuda.
 ↪memory_stats()['allocated_bytes.all.peak']/1024**3:.1f}GB")
    torch.cuda.reset_peak_memory_stats()
```

```
    gc.collect()
    torch.cuda.empty_cache()
```

[38]:
```
report_gpu()
```

```
Peak Memory Requirement: 3.9GB
```

So with `accum=1` the GPU used around 3GB RAM. Let's try `accum=2`:

[39]:
```
train('convnext_small_in22k', 128, epochs=1, accum=2, finetune=False)
report_gpu()
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

```
Peak Memory Requirement: 1.9GB
```

As you see, the RAM usage has now gone down to 2GB. It's not halved since there's other overhead involved (for larger models this overhead is likely to be relatively lower).

Let's try 4:

[40]:
```
train('convnext_small_in22k', 128, epochs=1, accum=4, finetune=False)
report_gpu()
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

```
Peak Memory Requirement: 1.4GB
```

The memory use is even lower!

[41]:
```
# Dump memory snapshot for viewing via https://pytorch.org/memory_viz
torch.cuda.memory._dump_snapshot("my_snapshot.pickle")
torch.cuda.memory._record_memory_history(None)
```

0.2 Checking memory use

We'll now check the memory use for each of the architectures and sizes we'll be training later, to ensure they all fit in 16GB RAM. For each of these, I tried `accum=1` first, and then doubled it any time the resulting memory use was over 16GB. As it turns out, `accum=2` was what I needed for every case.

First, `convnext_large`:

[42]:
```
# train('convnext_large.fb_in22k', 224, epochs=1, accum=2, finetune=False)
# report_gpu()
```

Here is another `convnext_large` with a larger input. This one is very close to going over the 16280MiB we've got on Kaggle!

[43]:
```
# train('convnext_large.fb_in22k', (320,240), epochs=1, accum=2, finetune=False)
# report_gpu()
```

Here's `vit_large`.

[44]:
```
# train('vit_large_patch16_224', 224, epochs=1, accum=2, finetune=False)
# report_gpu()
```

Then finally our `beit base and large` models:

[45]:
```
# # train('swinv2_large_window12_192_22k', 192, epochs=1, accum=2,␣
 ↪finetune=False)
# train('beitv2_base_patch16_224.in1k_ft_in1k', 224, epochs=1, accum=2,␣
 ↪finetune=False)
# report_gpu()
```

[46]:
```
# # train('swin_large_patch4_window7_224', 224, epochs=1, accum=2,␣
 ↪finetune=False)
# train('beitv2_large_patch16_224.in1k_ft_in22k', 224, epochs=1, accum=2,␣
 ↪finetune=False)
# report_gpu()
```

0.3 Running the models

Using the previous notebook, I tried a bunch of different architectures and preprocessing approaches on small models, and picked a few which looked good. We'll using a `dict` to list our the preprocessing approaches we'll use for each architecture of interest based on that analysis:

[47]:
```
res = 640,480
```

[48]:
```
# models = {
#     'convnext_large.fb_in22k': {
#         (Resize(res), 224),
#         (Resize(res), (320,224)),
#     }, 'vit_large_patch16_224': {
#         (Resize(480, method='squish'), 224),
#         (Resize(res), 224),
#     }, 'beitv2_base_patch16_224.in1k_ft_in1k': {
#         (Resize(480, method='squish'), 224),
#         (Resize(res), 224),
#     }, 'beitv2_large_patch16_224.in1k_ft_in22k': {
#         (Resize(480, method='squish'), 224),
#         (Resize(res), 224),
#     }
# }

models = {
```

```
    'convnext_small_in22k': {
        (Resize(480, method='squish'), 224),
    },
    'vit_small_patch16_224': {  # Add a VIT model
        (Resize(480, method='squish'), 224),
    }
}
```

We'll need to switch to using the full training set of course!

[49]:
```
trn_path = path/'train_images'
```

Now we're ready to train all these models. Remember that each is using a different training and validation set, so the results aren't directly comparable.

We'll append each set of TTA predictions on the test set into a list called `tta_res`.

[50]:
```
tta_res = []

for arch,details in models.items():
    for item,size in details:
        print('---',arch)
        print(size)
        print(item.name)
        tta_res.append(train(arch, size, item=item, accum=2)) #, epochs=1))
        gc.collect()
        torch.cuda.empty_cache()
```

```
--- convnext_small_in22k
224
Resize -- {'size': (480, 480), 'method': 'squish', 'pad_mode': 'reflection',
'resamples': (<Resampling.BILINEAR: 2>, <Resampling.NEAREST: 0>), 'p': 1.0}

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

--- vit_small_patch16_224
224
Resize -- {'size': (480, 480), 'method': 'squish', 'pad_mode': 'reflection',
'resamples': (<Resampling.BILINEAR: 2>, <Resampling.NEAREST: 0>), 'p': 1.0}
```

```
model.safetensors:   0%|          | 0.00/88.2M [00:00<?, ?B/s]
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

0.4 Ensembling

Since this has taken quite a while to run, let's save the results, just in case something goes wrong!

[51]:
```
save_pickle('tta_res.pkl', tta_res)
```

`Learner.tta` returns predictions and targets for each rows. We just want the predictions:

[52]:
```
tta_prs = first(zip(*tta_res))
```

Originally I just used the above predictions, but later I realised in my experiments on smaller models that `vit` was a bit better than everything else, so I decided to give those double the weight in my ensemble. I did that by simply adding the to the list a second time (we could also do this by using a weighted average):

[53]:
```
tta_prs += tta_prs[2:4]
```

An *ensemble* simply refers to a model which is itself the result of combining a number of other models. The simplest way to do ensembling is to take the average of the predictions of each model:

[54]:
```
avg_pr = torch.stack(tta_prs).mean(0)
avg_pr.shape
```

[54]:
```
torch.Size([3469, 10])
```

That's all that's needed to create an ensemble! Finally, we copy the steps we used in the last notebook to create a submission file:

[55]:
```
dls = ImageDataLoaders.from_folder(trn_path, valid_pct=0.2,␣
 ↪item_tfms=Resize(480, method='squish'),
    batch_tfms=aug_transforms(size=224, min_scale=0.75))
```

[56]:
```
idxs = avg_pr.argmax(dim=1)
vocab = np.array(dls.vocab)
ss = pd.read_csv(path/'sample_submission.csv')
```

```
ss['label'] = vocab[idxs]
ss.to_csv('subm.csv', index=False)
```

Now we can submit:

[57]:
```
if not iskaggle:
    from kaggle import api
    api.competition_submit_cli('subm.csv', 'part 3 v2', comp)
```

```
100%|      | 70.4k/70.4k [00:02<00:00, 33.4kB/s]
```

That's it – at the time of creating this analysis, that got easily to the top of the leaderboard! Here are the four submissions I entered, each of which was better than the last, and each of which was ranked #1:

Edit: Actually the one that got to the top of the leaderboard timed out when I ran it on Kaggle Notebooks, so I had to remove two of the runs from the ensemble. There's only a very small difference in accuracy however.

Going from bottom to top, here's what each one was:

1. `convnext_small` trained for 12 epochs, with TTA
2. `convnext_large` trained the same way
3. The ensemble in this notebook, with `vit` models not over-weighted
4. The ensemble in this notebook, with `vit` models over-weighted.

0.5 Conclusion

The key takeaway I hope to get across from this series so far is that you can get great results in image recognition using very little code and a very standardised approach, and that with a rigorous process you can improve in significant steps. Our training function, including data processing and TTA, is just half a dozen lines of code, plus another 7 lines of code to ensemble the models and create a submission file!

If you found this notebook useful, please remember to click the little up-arrow at the top to upvote it, since I like to know when people have found my work useful, and it helps others find it too. If you have any questions or comments, please pop them below – I read every comment I receive!

[58]:
```
# This is what I use to push my notebook from my home PC to Kaggle

if not iskaggle:
    push_notebook('isaacjohanziebarth', 'scaling-up-road-to-the-top-part-3',
                  title='Scaling Up: Road to the Top, Part 3',
                  file='10-scaling-up-road-to-the-top-part-3.ipynb',
                  competition=comp, private=False, gpu=True)
```

```
Kernel version 2 successfully pushed.  Please check progress at
https://www.kaggle.com/code/isaacjohanziebarth/scaling-up-road-to-the-top-part-3
```